Trame dei film vincitori ed eventi dello stesso anno

La nuova domanda di ricerca è: **le trame dei film vincitori dell'Oscar al miglior film presentano affinità tematiche con eventi avvenuti nell'anno della premiazione?**

Il punto di partenza resta il dataset del progetto. Il nuovo script `analisi_contesto_storico.py` legge `oscar_produzione_genere.json`, seleziona i vincitori e confronta la trama di ciascuno con una cronologia del suo anno. Il confronto precedente fra Wikipedia e LLM può rimanere come controllo dei dati; non risponde a questa nuova domanda.

Cosa fa lo script

1. Riusa titoli, registi, anni ed edizioni già presenti. Non riscrive gli input.
2. Recupera da Wikipedia trama, genere e anno di uscita. Controlla regia e anno prima di accettare la pagina, per ridurre gli errori dovuti ai titoli omonimi.
3. Legge la sezione **Eventi** della pagina Wikipedia dell'anno di riferimento, conservando testo, URL, ID e data della raccolta. Non include Nati/Morti o altre sezioni della cronologia.
4. Fornisce all'LLM soltanto quella trama e gli eventi di quell'anno. Il modello può proporre fino a tre affinità concrete, oppure nessuna.
5. Controlla che gli ID degli eventi provengano dall'input e che le citazioni compaiano nei testi. Le fonti sono aggiunte dal codice, non generate dal modello.
6. Salva JSON, CSV e riepilogo. Tutte le proposte richiedono revisione umana.

Il genere aiuta a leggere la scheda, ma non basta per associare un film a un fatto. Per esempio, la categoria «drammatico» non identifica un tema storico specifico.

Quale anno viene confrontato

Nel dataset originale `anno` è **l'anno degli Oscar**. Lo script lo chiama `anno_oscar` e lo usa come riferimento predefinito, in accordo con la richiesta. Non ricava l'anno di uscita sottraendo automaticamente uno: lo legge dalla scheda.

Un esempio: *Casablanca* è indicato nel dataset nell'edizione 1944, ma la scheda del film riporta 1942 come anno di uscita. Nella modalità predefinita il confronto avviene con gli eventi del 1944. L'ambientazione della trama è un'altra cosa ancora.

Per confrontare invece gli eventi dell'anno di uscita:

```
python3 analisi_contesto_storico.py prepara --wikipedia --riferimento uscita --output risulta
python3 analisi_contesto_storico.py analizza --input risultati_uscita/input_analisi.json --ou
```

Il criterio è l'anno solare intero. Un evento può essere successivo alla cerimonia o all'uscita: questa analisi non dimostra che abbia influenzato il film o il premio. Per studiare l'influenza servirebbero date precise di produzione/uscita, un diverso intervallo temporale e altre prove.

Avvio rapido con i dati già raccolti

Nel pacchetto è incluso `risultati_storici/input_analisi.json`: contiene i testi raccolti e permette di passare all'analisi senza ripetere lo scraping. La copertura effettiva è riportata in `COPERTURA_DATI.md`.

L'analisi via HTTP usa la libreria standard di Python. Per raccogliere nuovamente i dati e per caricare la chiave da `.env`, installare anche:

```
python3 -m pip install -r requirements_storico.txt
```

Configurare OpenRouter, come negli altri script del progetto:

```
export OPENROUTER_API_KEY="LA_TUA_CHIAVE"
python3 analisi_contesto_storico.py analizza --modello "SLUG_DEL_MODELLO"
```

Sostituire `SLUG_DEL_MODELLO` con un identificatore disponibile nel proprio account OpenRouter e compatibile con `response_format: json_object`. Il modello è esplicito, così non si presume che lo slug del vecchio script sia ancora disponibile. In alternativa all'export, installare `python-dotenv` e inserire la chiave nel file `.env` accanto allo script. Non caricare `.env` su GitHub.

La fase `analizza` invia i testi a OpenRouter e può consumare credito. La raccolta inclusa nel pacchetto e i test non hanno eseguito chiamate LLM a pagamento. Con i dati inclusi, un'esecuzione completa può fare una chiamata per ciascuno dei 98 film: conviene iniziare dalla prova su due film e scegliere un modello con una finestra di contesto adeguata ai testi. Documentazione del formato API: OpenRouter, chat completions.

Ricostruire l'input e iniziare con una prova piccola

```
python3 analisi_contesto_storico.py prepara --wikipedia --dal 1943 --al 1944 --output prova
python3 analisi_contesto_storico.py analizza --input prova/input_analisi.json --output prova
```

Per tutto il dataset:

```
python3 analisi_contesto_storico.py prepara --wikipedia
```

La prima fase non richiede chiavi API. Mantiene una cache locale delle pagine e attende fra le richieste. Gli errori di raccolta sono registrati, non trasformati in trame inventate. Alcuni titoli particolarmente ambigui possono richiedere una correzione manuale nell'input preparato, con relativa fonte.

Si può usare anche il CSV originale:

```
python3 analisi_contesto_storico.py prepara --input oscar_miglior_film.csv --wikipedia
```

Senza `--wikipedia`, la preparazione legge soltanto i dati locali. Per un input arricchito manualmente, aggiungere alle righe `trama`, `fonte_trama` ed eventualmente `anno_uscita`, mantenendo gli altri campi del dataset originale.

Usare un catalogo di eventi selezionato dalla studentessa

Per una ricerca universitaria è preferibile definire prima un insieme motivato di eventi politici, economici e sociali, poi confrontarlo con i film. Passare `--eventi eventi.json` alla preparazione. Il file deve contenere una lista:

```
[
  {
    "id_evento": "identificatore-univoco",
    "anno": 1944,
    "descrizione": "Testo documentato dell'evento da confrontare.",
    "fonte": "https://indirizzo-della-fonte.example/pagina"
  }
]
```

Questo è solo un esempio dello schema: sostituire testo e URL con dati reali. Gli eventi devono essere puntuali e associati a un anno; per processi lunghi come una guerra, registrare gli avvenimenti pertinenti anno per anno. `--eventi` sostituisce la raccolta delle cronologie, mentre `--wikipedia` può continuare a recuperare le trame.

Come leggere i risultati

Campo/esito	Significato
`COLLEGAMENTI_PROPOSTI`	Il modello propone una o più affinità da esaminare.
`FORTE`	Affinità specifica sostenuta da elementi espliciti nei due testi.
`PARZIALE`	Affinità tematica concreta ma indiretta.
`NESSUNA_ATTINENZA_EMERSA`	Nei testi e negli eventi forniti non è emerso un legame.
`DATI_INSUFFICIENTI`	Trama, anno o cronologia mancanti/insufficienti. Non equivale a nessun legame.
`ERRORE`	Problema di rete, risposta non valida o altro errore tecnico.

`analisi_storica.csv` contiene una riga per collegamento; un film senza proposte ha comunque una riga. Per contare i film usare gli ID univoci o i conteggi nel JSON, perché tre collegamenti non sono tre film.

Aprire il CSV con Excel o LibreOffice e verificare motivazioni, citazioni e pagine originali. **Salvare una copia come `revisione_umana.csv` prima di annotarla**: il CSV automatico viene rigenerato dallo script. Usare `giudizio_umano` per «confermato», «respinto» o «incerto» e `note_revisione` per il motivo.

Il JSON mantiene i testi, il prompt, il modello richiesto, il modello effettivo quando restituito dal provider e i consumi riportati dall'API. Le risposte JSON ricevute sono conservate in `risposte_llm/` per controllare le proposte.

L'analisi salva dopo ogni film. Rilanciare lo stesso comando riprende i record mancanti e riprova quelli in errore. Se cambiano input, prompt o modello, scegliere una nuova cartella `--output` per evitare di mescolare esecuzioni diverse. Errori di autenticazione, credito o configurazione interrompono le chiamate.

Limiti da scrivere nella relazione

- Le cronologie Wikipedia sono una selezione editoriale con una prospettiva geografica e culturale particolare. Non rappresentano tutti i fatti avvenuti. Gli eventi assenti non possono essere rilevati dall'analisi. Per l'anno corrente sono escluse le date future riconoscibili al momento della raccolta; eventuali annunci o descrizioni ambigue vanno comunque controllati.
- La trama riassume un'opera: non sostituisce la visione del film. Le pagine possono contenere errori e vanno controllate, soprattutto per i casi centrali.
- La pertinenza semantica è un giudizio interpretativo. I controlli del codice verificano anno, ID e presenza delle citazioni; non certificano la conclusione.
- Un film di guerra non si collega automaticamente a qualsiasi conflitto. Il modello deve motivare somiglianze specifiche e differenze.
- I vincitori da soli consentono un'analisi descrittiva. Per sostenere che gli Oscar privilegino determinati temi rispetto agli altri film, servirebbe almeno un confronto con i candidati non vincitori e un disegno di ricerca adeguato.
- Presentare frequenze e percentuali solo dopo la revisione, distinguendo i casi valutabili, mancanti e in errore. Non chiamare «correlazione statistica» un insieme di associazioni proposte dal modello.

Una formulazione adatta alla relazione: «L'LLM è stato usato come supporto alla codifica di affinità tematiche fra sinossi e cronologie annuali. Le proposte sono state controllate manualmente. L'analisi descrive corrispondenze nei testi selezionati e non dimostra un'influenza degli eventi sulle scelte dell'Academy».

Revisione del progetto esistente

Verifica effettuata sul commit `cabdd7b` del repository originale.

- `discordie.py` e `valutazione_openrouter.py` confrontano genere e produzione ottenuti con scraping e LLM: la tua lettura della funzione è corretta.
- Il file Wikipedia contiene 621 candidature, di cui 98 vincitori. Nei vincitori mancano 37 generi e tutte le 98 case di produzione; non ci sono trame. Il report preesistente indica infatti produzione non confrontabile nel 100% delle 621 righe. Va controllata l'estrazione prima di interpretare quella metrica.

- `scrapingllm.py` imposta `OUTPUT_JSON = "oscar_produzione_genere.json"`, lo stesso nome dell'output di scraping. Rischia di mescolare i dati se non si separano chiaramente i percorsi. La valutazione si aspetta invece il suffisso `_llm.json`.
- Il 1930 ha due edizioni, aprile e novembre. La nuova chiave comprende anno, edizione e titolo; anno da solo non identifica un'edizione.
- Il vecchio `requirements.txt` non elenca `requests` e `beautifulsoup4`, usati dagli scraper. Il nuovo modulo ha il proprio `requirements_storico.txt`.

Gli script e i dataset originali sono conservati nel pacchetto. La nuova analisi aggiunge un percorso separato, coerente anche con la fase di analisi qualitativa delle sinossi prevista dal documento di handoff già presente nel repository.

Verifiche tecniche e attribuzione

```
python3 -m unittest -v test_analisi_contesto_storico.py
```

I test usano un'API simulata: non verificano un modello reale. Sono state anche controllate l'estrazione su pagine pubbliche reali e la preparazione del dataset. Non è stata eseguita l'analisi LLM dell'intero corpus.

I testi raccolti da Wikipedia mantengono fonte e data; per riuso o pubblicazione rispettare la licenza di Wikipedia. La cache HTML serve per ripetere le verifiche locali e non è inclusa nel pacchetto.